

# System And Device Programming

## Cheat Sheet

### C language

#### Processes

- `pid_t getpid()`
- `pid_t getppid()`
- `pid_t fork (void)`
- `pid_t wait (int *statLoc)`
  - The status information are collected using macros defined in `<sys/wait.h>`
  - `WIFEXITED`, `WIFSIGNALED`, etc.
- `pid_t waitpid (pid_t pid, int *statLoc, int options)`
- `int execl (char *path, char *arg0, ..., (char *)0)`
- `int execlp (char *name, char *arg0, ..., (char *)0)`
- `int execl_e (char *path, char *arg0, ..., (char *)0, char *envp[])`
- `int execv (char *path, char *argv[])`
- `int execvp (char *name, char *argv[])`
- `int execve (char *path, char *argv[], char *envp[])`
- `int system (const char *string)`

#### Signals

- `void (*signal (int sig, void (*func)(int)))(int)`
  - `signal (SIGname, SIG_DFL)`
  - `signal (SIGname, SIG_IGN)`
  - `signal (SIGname, signalHandlerFunction)`
- `int kill (pid_t pid, int sig)`
- `int raise (int sig)`
- `int pause (void)`
- `unsigned int alarm (unsigned int seconds)`

#### Threads

- `int pthread_equal (pthread_t tid1, pthread_t tid2)`
- `pthread_t pthread_self (void)`
- `int pthread_create (pthread_t *tid, const pthread_attr_t *attr, void *(*startRoutine)(void *), void *arg)`
- `void pthread_exit (void *valuePtr)`
- `int pthread_join (pthread_t tid, void **valuePtr)`
- `int pthread_cancel (pthread_t tid)`
- `int pthread_detach (pthread_t tid)`
  - `pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED)`
  - `PTHREAD_CREATE_JOINABLE)`

#### Standard IO

- `int open (const char *path, int flags, mode_t mode)`
  -
- `int read (int fd, void *buf, size_t nbytes)`
- `int write (int fd, void *buf, size_t nbytes)`
- `off_t lseek (int fd, off_t offset, int whence)`
  - `whence = SEEK_SET, SEEK_CUR, or SEEK_END`

- `int close (int fd)`
- `int fstat (int fd, struct stat *sb)`
  - `struct stat {`
    - ⊖ `mode_t st_mode ; /* file type & mode */`
    - ⊖ `ino_t st_ino; /* i-node number */`
    - ⊖ `dev_t st_dev; /* device number */`
    - ⊖ `dev_t st_rdev; /* device number */`
    - ⊖ `...`
    - ⊖ `};`

### Non-Blocking IO

- `fd = open (name, O_WRONLY | ... | O_NONBLOCK )`
- `int fcntl (int fd, int cmd, ... /* int arg */ )`
  - `F_GETFL` or `F_SETFL`: Get/set file status flag
  - `F_DUPFD` or `F_DUPFD_CLOEXEC`: Duplicate an existing descriptor
  - `F_GETFD` or `F_SETFD`: Get/set file descriptor flags
  - `F_GETOWN` or `F_SETOWN`: Get/set asynchronous I/O ownership
  - `F_GETLK`, `F_SETLK`, or `F_SETLKW`: Get/set record locks

### File Locking

- `int fcntl(int fd, int cmd, struct flock *flockptr)`
  - `F_GETFL` or `F_SETFL`: Get/set file status flag
  - `F_DUPFD` or `F_DUPFD_CLOEXEC`: Duplicate an existing descriptor
  - `F_GETFD` or `F_SETFD`: Get/set file descriptor flags
  - `F_GETOWN` or `F_SETOWN`: Get/set asynchronous I/O ownership
  - `F_GETLK`, `F_SETLK`, or `F_SETLKW`: Get/set record locks
- `struct flock {`
  - `short l_type;`
  - `short l_whence;`
  - `off_t l_start;`
  - `off_t l_len;`
  - `pid_t l_pid;``};`

### Multiplexing IO

- `int select (int maxfdp1, fd_set *restrict readfds, fd_set *restrict writefds, fd_set *restrict exceptfds, struct timeval *restrict tvp)`
  - `tvp == NULL`, wait forever
  - `tvp->tv_sec == 0 && tvp->tv_usec == 0`, do not wait at all
  - `tvp->tv_sec != 0 || tvp->tv_usec != 0`, wait the specified number of seconds and microseconds
- `void FD_SET (int fd, fd_set *fdset)`
- `void FD_CLR (int fd, fd_set *fdset)`
- `void FD_ZERO (fd_set *fdset)`
- `int FD_ISSET (int fd, fd_set *fdset)`

### Asynchronous IO

- `struct aiocb {`
  - `int aio_fildes;`
  - `off_t aio_offset;`
  - `volatile void *aio_buf;`
  - `size_t aio_nbytes;`
  - `int aio_reqprio;`
  - `struct sigevent aio_sigevent;`

- ```

    int aio_lio_opcode;
};

```
- struct sigevent {
 int sigev\_notify;
 int sigev\_signo;
 union sigval sigev\_value;
 void (\*sigev\_notify\_function)(union sigval);
 pthread\_attr\_t \*sigev\_notify\_attributes;
};
  - int aio\_read(struct aiocb \*aiocb)
  - int aio\_write(struct aiocb \*aiocb)
  - int aio\_fsync (int op, struct aiocb \*aiocb)
  - int aio\_suspend(const struct aiocb \*const list[], int nent, const struct timespec \*timeout)
  - int aio\_cancel (int fd, struct aiocb \*aiocb)

### Memory Mapping

- void \*mmap (void \*addr, size\_t len, int prot, int flag, int fd, off\_t off)
  - prot = PROT\_READ, PROT\_WRITE, PROT\_EXEC, PROT\_NONE
  - flag = MAP\_FIXED, MAP\_SHARED, MAP\_PRIVATE
- void \*munmap (void \*addr, size\_t len)

### Semaphores

- int sem\_init (sem\_t \*sem, int pshared, unsigned int value)
- int sem\_wait (sem\_t \*sem)
- int sem\_trywait (sem\_t \*sem)
- int sem\_post (sem\_t \*sem)
- int sem\_getvalue (sem\_t \*sem, int \*valP)
- int sem\_destroy (sem\_t \*sem)

### Mutexes

- int pthread\_mutex\_init (pthread\_mutex\_t \*mutex, const pthread\_mutexattr\_t \*attr)
  - PTHREAD\_MUTEX\_INITIALIZER
- int pthread\_mutex\_lock (pthread\_mutex\_t \*mutex)
- int pthread\_mutex\_trylock (pthread\_mutex\_t \*mutex)
- int pthread\_mutex\_unlock (pthread\_mutex\_t \*mutex)
- int pthread\_mutex\_destroy (pthread\_mutex\_t \*mutex)

### Reader-Writer Locks

- int pthread\_rwlock\_init (pthread\_rwlock\_t \*restrict rwlock, const pthread\_rwlockattr\_t \*restrict attr)
- int pthread\_rwlock\_rdlock (pthread\_rwlock\_t \*rwlock)
- int pthread\_rwlock\_wrlock (pthread\_rwlock\_t \*rwlock)
- int pthread\_rwlock\_unlock (pthread\_rwlock\_t \*rwlock)
- int pthread\_rwlock\_tryrdlock(pthread\_rwlock\_t \*rwlock)
- int pthread\_rwlock\_trywrlock(pthread\_rwlock\_t \*rwlock)
- int pthread\_rwlock\_timerdlock(pthread\_rwlock\_t \*restrict rwlock, const struct timespec \*restrict tsptr)
- int pthread\_rwlock\_timedwrlock(pthread\_rwlock\_t \*restrict rwlock, const struct timespec \*restrict tsptr)
- int pthread\_rwlock\_destroy (pthread\_rwlock\_t \*rwlock)

## Condition Variables

- `int pthread_cond_init (pthread_cond_t *restrict cond, const pthread_condattr_t *restrict attr)`
- `int pthread_cond_wait (pthread_cond_t *restrict cond, pthread_mutex_t *restrict mutex)`
- `int pthread_cond_timedwait (pthread_cond_t *restrict cond, pthread_mutex_t *restrict mutex, const struct timespec *restrict ts_ptr)`
- `int pthread_cond_signal (pthread_cond_t *cond)`
- `int pthread_cond_broadcast (pthread_cond_t *cond)`
- `int pthread_cond_destroy (pthread_cond_t *cond)`

## Spin Locks

- `int pthread_spin_init (pthread_spinlock_t *lock, int pshared)`
- `int pthread_spin_lock (pthread_spinlock_t *lock)`
- `int pthread_spin_trylock (pthread_spinlock_t *lock)`
- `int pthread_spin_unlock (pthread_spinlock_t *lock)`
- `int pthread_spin_destroy (pthread_spinlock_t *lock)`

## Barriers

- `int pthread_barrier_init (pthread_barrier_t *restrict barrier, const pthread_barrierattr_t *restrict attr, unsigned int count)`
- `int pthread_barrier_wait (pthread_barrier_t *barrier)`
- `int pthread_barrier_destroy (pthread_barrier_t *barrier)`

## Pipes

- `int pipe (int file_descr[2])`

## FIFOs

- `int mkfifo (const char *path, mode_t mode)`
- `int mkfifoat (int fd, const char *path, mode_t mode)`

## Keys

- `key_t ftok (const char *path, int id)`

## Message Queues

- `int msgget (key_t key, int flag)`
- `int msgctl (int msqid, int cmd, struct msqid_ds *buf)`
  - `cmd = IPC_STAT, IPC_SET, IPC_RMID`
- `int msgsnd (int msqid, const void *ptr, size_t nbytes, int flag)`
- `ssize_t msgrcv (int msqid, void *ptr, size_t nbytes, long type, int flag)`

## Shared Memory

- `int shmget (key_t key, size_t size, int flag)`
- `int shmctl (int shmid, int cmd, struct shmid_ds *buf)`
- `void *shmat (int shmid, const void *addr, int flag)`
- `int shmdt (const void *addr)`

## Sockets

- `include <sys/socket.h>`
- `int socket(int domain, int type, int protocol)`

- `int connect(int sockfd, const struct sockaddr *addr, socklen_t len)`
- `int bind(int sockfd, const struct sockaddr *addr, socklen_t len)`
- `int listen(int sockfd, int backlog)`
- `int accept(int sockfd, struct sockaddr *addr, socklen_t *restrict len)`

## C++ Language

### Streams

- **Input Streams (`std::istream`)**
  - Constructors:
    - `istream()`
    - `istream(std::streambuf* sb)`
  - Member Functions:
    - `int get()`
    - `std::istream& getline(char* str, std::streamsize count)`
    - `std::istream& ignore(std::streamsize count = 1, int delim = EOF)`
    - `int peek()`
    - `std::istream& operator>>(T& value)`
- **Output Streams (`std::ostream`)**
  - Constructors
    - `ostream()`
    - `ostream(std::streambuf* sb)`
  - Member Functions
    - `std::ostream& put(char c)`
    - `std::ostream& write(const char* s, std::streamsize count)`
    - `flush()`
    - `std::ostream& operator<<(T value)`
- **File Streams (`std::ifstream`, `std::ofstream`, `std::fstream`)**
  - **Constructors**
    - `ifstream()`
    - `ifstream(const char* filename, std::ios_base::openmode mode = std::ios_base::in)`
    - `ofstream()`
    - `ofstream(const char* filename, std::ios_base::openmode mode = std::ios_base::out)`
    - `fstream()`
    - `fstream(const char* filename, std::ios_base::openmode mode = std::ios_base::in | std::ios_base::out)`
    - `std::ios_base::in, out, binary, ate, app, trunc`
  - **Member Functions**
    - `void open(const char* filename, std::ios_base::openmode mode)`
    - `bool is_open()`
    - `void close()`

### Containers

- **`std::vector<T>`**
  - Constructors
    - `vector()`, `vector(size_type count)`, `vector(size_type count, const T& value)`, `vector(InputIt first, InputIt last)`, ...
  - Member Functions
    - `size_type size()`, `bool empty()`, `T& operator[](size_type pos)`, `void push_back(const T& value)`, `template <class... Args> void`

```

        emplace_back(Args&&... args), void pop_back(), void clear(),
        iterator begin(), iterator end(), iterator
        insert(const_iterator pos, const T& value), iterator
        erase(const_iterator pos), ...

```

- **std::deque<T>**

- Constructors
  - deque(), deque(size\_type count), deque(size\_type count, const T& value), deque(InputIt first, InputIt last), ...
- Member Functions
  - size\_type size(), bool empty(), T& operator[](size\_type pos), void push\_back(const T& value), template <class... Args> void emplace\_back(Args&&... args), void pop\_back(), void push\_front(const T& value), template <class... Args> void emplace\_front(Args&&... args), void pop\_front(), void clear(), iterator begin(), iterator end(), iterator insert(const\_iterator pos, const T& value), iterator erase(const\_iterator pos), ...

- **std::list<T>**

- Constructors
  - list(), list(size\_type count), list(size\_type count, const T& value), list(InputIt first, InputIt last), ...
- Member Functions
  - size\_type size(), bool empty(), void push\_back(const T& value), template <class... Args> void emplace\_back(Args&&... args), void pop\_back(), void push\_front(const T& value), template <class... Args> void emplace\_front(Args&&... args), void pop\_front(), void clear(), iterator begin(), iterator end(), iterator insert(const\_iterator pos, const T&value), iterator erase(const\_iterator pos), ...

- **std::forward\_list<T>**

- Constructors
  - forward\_list(), forward\_list(size\_type count), forward\_list(size\_type count, const T& value), forward\_list(InputIt first, InputIt last), ...
- Member Functions
  - bool empty(), void clear(), iterator begin(), iterator end(), iterator insert\_after(const\_iterator pos, const T& value), template <class... Args> iterator emplace\_after(const\_iterator pos, Args&&... args), iterator erase\_after(const\_iterator pos), ...

- **std::array<T, N>**

- Constructors
  - array(), array(const T& value), array(std::initializer\_list<T> ilist), ...
- Member Functions
  - size\_type size(), bool empty(), void fill(const T& value), iterator begin(), iterator end(), reference at(size\_type pos), reference operator[](size\_type pos), ...

- **std::string**

- Constructors
  - string()
  - string(const char\* s)
  - string(const char\* s, size\_type count)
  - string(const std::string& str)
  - string(size\_type count, char ch)

- ...
- o Member Functions
  - size\_type size()
  - size\_type length()
  - bool empty()
  - void clear()
  - const char\* c\_str()
  - const char\* data()
  - void push\_back(char ch)
  - void pop\_back()
  - void resize(size\_type count)
  - size\_type capacity()
  - void reserve(size\_type new\_cap)
  - void append(const std::string& str)
  - std::string& operator+=(const std::string& str)
  - void insert(size\_type pos, const std::string& str)
  - void erase(size\_type pos, size\_type count)
  - void replace(size\_type pos, size\_type count, const std::string& str)
  - std::string substr(size\_type pos, size\_type count)
  - size\_type find(const std::string& str, size\_type pos)
  - size\_type rfind(const std::string& str, size\_type pos)
  - int compare(const std::string& str)
- **std::set<Key, Compare, Allocator>**
  - o Constructors
    - set(), set(InputIt first, InputIt last), set(std::initializer\_list<T> ilist), ...
  - o Member Functions
    - size\_type size() const, bool empty() const, iterator begin() const, iterator end() const, std::pair<iterator, bool> insert(const value\_type& value), iterator erase(iterator pos), ...
- **std::map<Key, T, Compare, Allocator>**
  - o Constructors
    - map(), map(InputIt first, InputIt last), map(std::initializer\_list<value\_type> ilist), ...
  - o Member Functions
    - size\_type size() const, bool empty() const, iterator begin() const, iterator end() const, mapped\_type& operator[](const key\_type& key), std::pair<iterator, bool> insert(const value\_type& value), iterator erase(iterator pos), ...
- **std::unordered\_set<Key, Hash, KeyEqual, Allocator>**
  - o Constructors
    - unordered\_set(), unordered\_set(size\_type bucket\_count), unordered\_set(size\_type bucket\_count, const Hash& hash, const KeyEqual& equal, const Allocator& alloc), ...
  - o Member Functions
    - size\_type size() const, bool empty() const, iterator begin() const, iterator end() const, std::pair<iterator, bool> insert(const value\_type& value), iterator erase(iterator pos), ...
- **std::unordered\_map<Key, T, Hash, KeyEqual, Allocator>**
  - o Constructors

- `unordered_map()`, `unordered_map(size_type bucket_count)`, `unordered_map(size_type bucket_count, const Hash& hash, const KeyEqual& equal, const Allocator& alloc)`, ...
- o Member Functions
  - `size_type size() const`, `bool empty() const`, `iterator begin() const`, `iterator end() const`, `mapped_type& operator[](const key_type& key)`, `std::pair<iterator, bool> insert(const value_type& value)`, `iterator erase(iterator pos)`, ...

## Algorithms

### • Non-modifying Sequence Operations

- o `InputIt find(InputIt first, InputIt last, const T& value)`
  - Finds the first occurrence of a value in a range
- o `DifferenceType count(InputIt first, InputIt last, const T& value)`
  - Counts the number of occurrences of a value in a range
- o `bool all_of(InputIt first, InputIt last, UnaryPredicate p)`
  - Checks if a predicate is true for all elements in a range
- o `bool any_of(InputIt first, InputIt last, UnaryPredicate p)`
  - Checks if a predicate is true for any element in a range
- o `bool none_of(InputIt first, InputIt last, UnaryPredicate p)`
  - Checks if a predicate is false for all elements in a range
- o `InputIt find_if(InputIt first, InputIt last, UnaryPredicate p)`
  - Finds the first element in a range that satisfies a predicate
- o `InputIt find_if_not(InputIt first, InputIt last, UnaryPredicate p)`
  - Finds the first element in a range that does not satisfy a predicate
- o `DifferenceType count_if(InputIt first, InputIt last, UnaryPredicate p)`
  - Counts the number of elements in a range that satisfy a predicate
- o `bool equal(InputIt1 first1, InputIt1 last1, InputIt2 first2)`
  - Checks if two ranges are equal

### • Modifying Sequence Operations

- o `OutputIt copy(InputIt first, InputIt last, OutputIt d_first)`
  - Copies a range of elements from the source range defined by `[first, last)` to the destination range beginning at `d_first`
- o `OutputIt copy_if(InputIt first, InputIt last, OutputIt d_first, UnaryPredicate p)`
  - Copies elements from the source range defined by `[first, last)` to the destination range beginning at `d_first`, selectively copying only those that satisfy the predicate `p`
- o `OutputIt copy_n(InputIt first, Size count, OutputIt d_first)`
  - Copies `count` elements from the source range beginning at `first` to the destination range beginning at `d_first`
- o `BidirIt copy_backward(BidirIt first, BidirIt last, BidirIt d_last)`
  - Copies a range of elements from the source range defined by `[first, last)` to the destination range beginning at `d_last`, preserving the order of elements
- o `OutputIt move(InputIt first, InputIt last, OutputIt d_first)`



- Moves a range of elements from the source range defined by [first, last) to the destination range beginning at d\_first
- o BidirIt move\_backward(BidirIt first, BidirIt last, BidirIt d\_last)
  - Moves a range of elements from the source range defined by [first, last) to the destination range beginning at d\_last, preserving the order of the elements
- o void fill(ForwardIt first, ForwardIt last, const T& value)
  - Assigns the value value to every element in the range [first, last)
- o OutputIt fill\_n(OutputIt first, Size count, const T& value)
  - Assigns the value value to count elements starting from the iterator first
- o OutputIt transform(InputIt1 first1, InputIt1 last1, InputIt2 first2, OutputIt d\_first, BinaryOperation binary\_op)
  - Applies the binary operation binary\_op to each pair of elements from two input ranges [first1, last1) and stores the result in the corresponding element of the output range beginning at d\_first
- o void generate(ForwardIt first, ForwardIt last, Generator g)
  - Assigns the result of invoking the generator function g to each element in the range [first, last)
- o OutputIt generate\_n(OutputIt first, Size count, Generator g)
  - Assigns the result of invoking the generator function g to count elements starting from the iterator first
- o ForwardIt remove(ForwardIt first, ForwardIt last, const T& value)
  - Removes all elements equal to value from the range [first, last), shifting all the elements after the removal range to fill the gap
- o ForwardIt remove\_if(ForwardIt first, ForwardIt last, UnaryPredicate p)
  - Removes all elements satisfying the predicate p from the range [first, last), shifting all the elements after the removal range to fill the gap
- o void replace(ForwardIt first, ForwardIt last, const T& old\_value, const T& new\_value)
  - Replaces all elements equal to old\_value in the range [first, last) with new\_value
- o void replace\_if(ForwardIt first, ForwardIt last, UnaryPredicate p, const T& new\_value)
  - Replaces all elements satisfying the predicate p in the range [first, last) with new\_value
- o void reverse(BidirIt first, BidirIt last)
  - Reverses the order of elements in the range [first, last).
- o ForwardIt rotate(ForwardIt first, ForwardIt n\_first, ForwardIt last)
  - Rotates the order of elements in the range [first, last) such that the element pointed by n\_first becomes the first element
- o ForwardIt unique(ForwardIt first, ForwardIt last)
  - Removes all consecutive duplicate elements from the range [first, last), only leaving the first occurrence of each unique element
- o OutputIt unique\_copy(InputIt first, InputIt last, OutputIt

d\_first)

- Copies the elements from the input range defined by [first, last) to the output range beginning at d\_first, removing consecutive duplicate elements

- **Sorting and Related Operations**

- o void sort(RandomIt first, RandomIt last)
  - Sorts the elements in the range [first, last) in ascending order
- o void partial\_sort(RandomIt first, RandomIt middle, RandomIt last)
  - Rearranges the elements in the range [first, last) such that the range [first, middle) contains the sorted elements in ascending order. The order of the remaining elements in the range [middle, last) is unspecified
- o OutputIt partial\_sort\_copy(InputIt first, InputIt last, OutputIt d\_first, OutputIt d\_last)
  - Copies the smallest elements from the input range [first, last) to the output range beginning at d\_first. The number of copied elements is determined by the smaller of the range [first, last) or the range [d\_first, d\_last)
- o void stable\_sort(RandomIt first, RandomIt last)
  - Sorts the elements in the range [first, last) in ascending order while maintaining the relative order of the elements with equivalent values
- o void nth\_element(RandomIt first, RandomIt nth, RandomIt last)
  - Rearranges the elements in the range [first, last) such that the element pointed to by nth is in its sorted position. The order of the elements before nth and after nth is unspecified
- o bool is\_sorted(ForwardIt first, ForwardIt last)
  - Checks if the elements in the range [first, last) are sorted in ascending order. Returns true if the range is sorted, false otherwise
- o ForwardIt is\_sorted\_until(ForwardIt first, ForwardIt last)
  - Finds the iterator pointing to the first element in the range [first, last) that is not sorted in ascending order
- o bool binary\_search(ForwardIt first, ForwardIt last, const T& value)
  - Checks if the value exists in the sorted range [first, last). Returns true if the value is found, false otherwise
- o ForwardIt lower\_bound(ForwardIt first, ForwardIt last, const T& value)
  - Finds the first position in the sorted range [first, last) where the value can be inserted without violating the sorting order
- o ForwardIt upper\_bound(ForwardIt first, ForwardIt last, const T& value)
  - Finds the last position in the sorted range [first, last) where the value can be inserted without violating the sorting order
- o std::pair<ForwardIt, ForwardIt> equal\_range(ForwardIt first, ForwardIt last, const T& value)
  - Finds a range of elements in the sorted range [first, last) that are equal to the given value. Returns a pair of iterators representing the subrange

- o `OutputIt merge(InputIt1 first1, InputIt1 last1, InputIt2 first2, InputIt2 last2, OutputIt d_first)`
  - Merges two sorted ranges `[first1, last1)` and `[first2, last2)` into a single sorted range beginning at `d_first`
- o `void inplace_merge(BidirIt first, BidirIt middle, BidirIt last)`
  - Merges two consecutive sorted ranges `[first, middle)` and `[middle, last)` into a single sorted range

#### • Partitioning Operations

- o `ForwardIt partition(ForwardIt first, ForwardIt last, UnaryPredicate p)`
  - Rearranges the elements in the range `[first, last)` such that all elements for which the predicate `p` returns true precede the elements for which the predicate returns false. Returns an iterator to the first element of the second group
- o `ForwardIt stable_partition(ForwardIt first, ForwardIt last, UnaryPredicate p)`
  - Rearranges the elements in the range `[first, last)` such that all elements for which the predicate `p` returns true precede the elements for which the predicate returns false. The relative order of the elements within each group is preserved. Returns an iterator to the first element of the second group
- o `OutputIt1 partition_copy(InputIt first, InputIt last, OutputIt1 d_first_true, OutputIt2 d_first_false, UnaryPredicate p)`
  - Copies the elements from the range `[first, last)` to two different output ranges: `[d_first_true, ...)` for the elements for which the predicate `p` returns true, and `[d_first_false, ...)` for the elements for which the predicate returns false. Returns a pair of output iterators
- o `ForwardIt partition_point(ForwardIt first, ForwardIt last, UnaryPredicate p)`
  - Finds the partition point in the sorted range `[first, last)`, i.e., the first element that does not satisfy the predicate `p`
- o `bool is_partitioned(ForwardIt first, ForwardIt last, UnaryPredicate p)`
  - Checks if the elements in the range `[first, last)` are partitioned according to the predicate `p`
- o `OutputIt1 partition_copy(InputIt first, InputIt last, OutputIt1 d_first_true, OutputIt2 d_first_false, UnaryPredicate p)`
  - Copies the elements from the range `[first, last)` to two different output ranges: `[d_first_true, ...)` for the elements for which the predicate `p` returns true, and `[d_first_false, ...)` for the elements for which the predicate returns false. Returns a pair of output iterators

#### • Numeric Algorithms

- o `T accumulate(InputIt first, InputIt last, T init)`
  - Computes the sum of the elements in the range `[first, last)` starting from the initial value `init`
- o `T reduce(InputIt first, InputIt last)`
  - Reduces the elements in the range `[first, last)` using the `+` operator. Returns the sum of all elements
- o `T inner_product(InputIt1 a_first, InputIt1 a_last, InputIt2 b_first, T init)`

- Computes the inner product of two ranges `[a_first, a_last)` and `[b_first, ...)`, optionally starting with the initial value `init`
- o `OutputIt partial_sum(InputIt first, InputIt last, OutputIt d_first)`
  - Computes the partial sums of the elements in the range `[first, last)` and stores them in the range starting from `d_first`
- o `OutputIt inclusive_scan(InputIt first, InputIt last, OutputIt d_first)`
  - Computes the inclusive scan of the elements in the range `[first, last)` and stores the results in the range starting from `d_first`. Each element in the output range is the reduction of the corresponding elements in the input range
- o `OutputIt exclusive_scan(InputIt first, InputIt last, OutputIt d_first, T init)`
  - Computes the exclusive scan of the elements in the range `[first, last)` using the initial value `init` and stores the results in the range starting from `d_first`. Each element in the output range is the reduction of the corresponding elements in the input range and the previous element
- o `T transform_reduce(InputIt1 a_first, InputIt1 a_last, InputIt2 b_first, T init, BinaryOp op1, BinaryOp op2)`
  - Combines the operations of transforming and reducing the elements in two ranges `[a_first, a_last)` and `[b_first, ...)`, starting with the initial value `init` and using the binary operators `op1` and `op2`
- o `OutputIt transform_exclusive_scan(InputIt first, InputIt last, OutputIt d_first, T init, BinaryOp op, UnaryOp unary_op)`
  - Computes the exclusive scan of the elements in the range `[first, last)` using the initial value `init` and stores the results in the range starting from `d_first`. Each element in the output range is the result of applying the binary operation `op` to the corresponding elements in the input range, followed by the unary operation `unary_op`
- o `OutputIt transform_inclusive_scan(InputIt first, InputIt last, OutputIt d_first, BinaryOp op, UnaryOp unary_op)`
  - Computes the inclusive scan of the elements in the range `[first, last)` and stores the results in the range starting from `d_first`. Each element in the output range is the result of applying the binary operation `op` to the corresponding elements in the input range, followed by the unary operation `unary_op`
- o `void iota(ForwardIt first, ForwardIt last, T value)`
  - Assigns consecutive values to the elements in the range `[first, last)`, starting from the initial value `value` and incrementing by 1

## Lambda Expressions

```
[capture_list](parameters) -> return_type { lambda_body }
```

## Threads and synchronization

### • Thread Creation

- o `std::thread (#include <thread>)`
  - Creation: `std::thread threadObj(Function, args...)`

- o Lambda Functions
    - `std::thread threadObj([](){ /* Thread logic */ })`
- **Thread Management**
  - o Joining Threads
    - `threadObj.join()`
      - Waits for the thread to finish its execution
  - o Detaching Threads
    - `threadObj.detach()`
      - Separates the thread from the calling thread, allowing it to run independently
  - o Checking if a Thread is Joinable
    - `threadObj.joinable()`
      - Checks if the thread is joinable (can be joined or detached)
  - o Sleep
    - `std::this_thread::sleep_for(std::chrono::milliseconds(100))`
      - Sleep for a specified duration
  - o Thread ID
    - `std::thread::id threadId = std::this_thread::get_id()`
      - Get the ID of the current thread
  - o Hardware Concurrency
    - `unsigned int numThreads = std::thread::hardware_concurrency()`
      - Get the number of supported concurrent threads
- **Thread Synchronization**
  - `#include <mutex> (<shared_mutex>)`
    - `std::mutex mutexObj`
  - `mutexObj.lock():`
    - Acquires the lock on the mutex
  - `mutexObj.unlock()`
    - Releases the lock on the mutex
  - `std::recursive_mutex recursiveMutexObj`
    - Recursive mutex object
  - `std::shared_mutex sharedMutexObj`
    - Shared mutex object for shared ownership
  - `std::unique_lock (<include <mutex>)`
    - Creation: `std::unique_lock<std::mutex> u_lock(mutexObj)`
  - `u_lock.lock()`
    - Explicitly acquire the lock
  - `u_lock.unlock()`
    - Explicitly release the lock
  - `u_lock.try_lock()`
    - Try to acquire the lock, returns immediately
  - `std::lock_guard (<include <mutex>)`
    - Creation: `std::lock_guard<std::mutex> lock(mutexObj)`
- **RAII locks**
  - `std::shared_lock<std::shared_mutex> sharedLock(sharedMutexObj)`
    - Shared lock, allows multiple readers
  - `std::scoped_lock<std::mutex, std::mutex> scopedLock(mutexObj1, mutexObj2)`
    - Scoped lock, locks multiple mutexes

- `std::condition_variable` (`#include <condition_variable>`)
  - Creation: `std::condition_variable cv`
- `cv.wait(unique_lock)`
  - Blocks the thread until notified
- `cv.notify_one()`
  - Notifies one waiting thread
- `cv.notify_all()`
  - Notifies all waiting threads
- **Barriers**
  - `#include <barrier>`
  - `std::barrier barrierObj(count)`
    - Create a barrier with the specified count
  - `barrierObj.arrive_and_wait()`
    - Wait at the barrier and synchronize all threads
  - `barrierObj.arrive_and_drop()`
    - Wait at the barrier and release one waiting thread
- **Asynchronous execution (promise and future)**
  - `#include <future>`
  - `std::future<int> fut = std::async(std::launch::async, func)`
    - Launch a function asynchronously and get a future
  - `int result = fut.get()`
    - Wait for the future and retrieve the result
- **Promise and Future (`#include <future>`)**
  - `std::promise<int> promiseObj`
    - Promise object for communication between threads
  - `std::future<int> futureObj = promiseObj.get_future()`
    - Get a future associated with the promise
  - `promiseObj.set_value(42)`
    - Set the value in the promise
  - `int value = futureObj.get()`
    - Get the value from the future